

2026
Coding Standards
Uni Textbook Marketplace

For **Project Day**

Presented by:

NexusDev

Coding Standards

Project:	Uni Textbook Marketplace
Team:	NexusDev
Client:	Agile Bridge
University:	University of Pretoria - COS 301 Software Engineering 2026
Document version:	3.0 - Demo 4
Last updated:	28 September 2026

The Team

Team Member	Student number
1. Tiego Mokwena*	22496336
2. Gift Mohuba	23545527
3. Neo Bosoga	23591732
4. Josh Kretschmer	22883925

Contents

1 Introduction **3**

2 Repository File Structure **4**
2.1 Module Structure (Backend) . . . 5
2.2 Route Structure (Frontend) . . . 6

3 Naming Conventions **7**

4 TypeScript and Formatting Standards **8**
4.1 Compiler Configuration . . . 8
4.2 Linting . . . 8
4.3 Static Analysis . . . 8

5 Backend Standards (NestJS) **9**
5.1 Layering . . . 9
5.2 Guards and Authorization . . . 9
5.3 Data Transfer Objects . . . 9
5.4 Database Migrations . . . 9
5.5 Event-Driven Communication Between Modules . . . 10
5.6 Concurrency and Transactions . . . 10
5.7 External Services and Graceful Degradation . . . 11
5.8 Algorithmic Code . . . 11

6 Frontend Standards (Next.js) **12**
6.1 Styling . . . 12
6.2 Authentication State . . . 12
6.3 API Calls . . . 12
6.4 Component Organisation . . . 12
6.5 Interactive and Media Components . . . 12

7 Git and Version Control **14**
7.1 Branching Model . . . 14
7.2 Branch Protection . . . 14
7.3 Commit Messages . . . 14

8 Testing Standards **15**

9 Environment Variables and Secrets **17**

1. Introduction

This document lays out the coding conventions we use across the Uni Textbook Marketplace project. The main point is just to keep things consistent, since four people are writing code and it's easy for that to turn into a mess if everyone does their own thing. It should also help whoever's marking this actually follow the repo without us having to explain everything in person. This version covers the codebase as it stands for Demo 4, including the three wow factors added in Sprint 6 (AI-assisted listing creation, real-time auctions, and the bundle optimizer), and the extra conventions those features forced on us.

Most of what's below isn't just theory, a lot of it is actually enforced by ESLint, SonarCloud, and our CI pipeline (more detail on that in the SAS document), so if you break one of the bigger rules the pull request literally won't merge.

2. Repository File Structure

We're using a monorepo, so it's one Git repo with the backend and frontend sitting in their own top-level folders, plus the CI/CD config.

```
Uni-Textbook-Marketplace/
  .github/
    workflows/
      ci.yml          # lint, test, coverage, on every push and PR
      deploy.yml      # build and deploy, branch-conditional
  backend/
    src/
      auth/           # Controller, service, OTP, guards, strategies, DTOs
      listings/       # Listing CRUD, search, edit/withdraw, status
      admin/          # Review dashboard, audit log, admin events
      cases/          # Moderation cases and appeals
      reports/        # Listing/user reports
      modules/        # Module search-as-you-type
      books/          # Book/catalogue metadata
      messaging/      # Buyer-seller messaging (Firestore-backed)
      notifications/  # Notification service, stream, event listeners
      saved_search/   # Saved searches and match alerts
      wishlist/       # Wishlist feature
      auction/        # Wow factor: real-time auctions (bids, closing)
      bundles/        # Wow factor: multi-book bundle optimizer
      vision/         # Wow factor: OCR text extraction, book matching
      azure/          # Blob upload controller and service
      email/          # Email provider and templates (OTP, alerts)
      firebase/       # Firebase Admin SDK init
      shared/         # Shared constants/utilities across modules
      database/
        entities/     # TypeORM entity classes
        migrations/   # TypeORM migration files, flat, no nesting
        schema/       # Reference SQL schema (incl. test DB)
        seeds/        # Idempotent seed scripts and seed images
      data-source.ts  # TypeORM CLI data source
      main.ts         # Application bootstrap, CORS config
      app.module.ts   # Root module, TypeOrmModule config
    scripts/          # One-off tooling (e.g. module scraper)
    test/             # Integration and e2e tests (Supertest)
    Dockerfile
    Dockerfile.dev
```

```
  package.json
frontend/
  src/
    app/              # Next.js App Router, one folder per route
    components/        # Reusable UI, grouped by feature
      ui/               # Base component library (Button, Modal...)
      listings/         # Listing form, card, AI photo capture
      auction/          # Auction card
      bundles/          # Bundle optimizer, module picker, results
      messaging/, notifications/, admin/, auth/, wishlist/
    context/             # React context providers (e.g. AuthContext)
    hooks/                # Custom React hooks
    lib/                  # API client, one *.api.ts per backend area
    providers/              # Theme and other app-wide providers
    types/                  # Shared TypeScript types
    utils/                  # Frontend utility functions
  cypress/               # Cypress e2e specs, fixtures, support
  public/                 # Static images and media
  Dockerfile
  Dockerfile.dev
  package.json
database/
  schema/             # Reference DB schema
docs/
  Demo_1/ ... Demo_4/   # Per-demo SRS, SAS, standards, manuals
  OpenAPI/               # openapi.yaml service contracts
  images/                 # Shared logos and diagrams
docker-compose.yml  # Local dev environment
.env                  # Local environment variables, not committed
```

Tests sit next to the code they cover wherever it makes sense: `*.spec.ts` unit tests beside their service or utility in the backend, `tests/` subfolders beside frontend components, hooks and utils, and anything needing the real database or the full HTTP stack goes in `backend/test/`.

2.1 Module Structure (Backend)

Each backend feature is its own self-contained NestJS module. Modules aren't allowed to reach into another module's internals. If two modules need to share something, it goes through a shared provider, and if one module just needs to tell another that something happened, it emits an event instead of calling it directly (see the event-driven section

below). A typical module looks like this:

```
listings/
  dto/
    create-listing.dto.ts
    listing-filter.dto.ts
  events/
    edit.event.ts
  listings.controller.ts
  listings.service.ts
  listings.service.spec.ts
  listings.module.ts
```

Not every module needs every folder. `dto/` appears when the module accepts request bodies, `events/` when it emits domain events, and a `*.spec.ts` sits next to any service or utility with real logic in it.

2.2 Route Structure (Frontend)

We're using the Next.js App Router, so every route gets its own folder under `src/app`, named after the URL it serves (e.g. `src/app/saved-searches/page.tsx` is `/saved-searches`, and `src/app/auction/bid/page.tsx` is `/auction/bid`). Anything only used on that one page lives next to it, and anything reused across pages goes into `src/components`.

3. Naming Conventions

What	Convention	Example
TypeScript classes, interfaces, enums	PascalCase	`ListingService`, `CreateListingDto`
Variables, functions, methods	camelCase	`getPendingListings()`
NestJS file names	kebab-case, dot-suffixed by role	`listings.controller.ts`, `listings.service.ts`, `create-listing.dto.ts`
TypeORM entity files	kebab/snake, `.entity.ts` suffix	`auction.entity.ts`, `saved_search.entity.ts`
Domain event files	`.event.ts` suffix	`edit.event.ts`, `message.event.ts`
Test files	`.spec.ts` (unit, integration), `.e2e-spec.ts`, `.test.tsx` (frontend), `.cy.ts` (Cypress)	`bundles.service.spec.ts`, `auth.e2e-spec.ts`, `login.cy.ts`
React component files	PascalCase	`NavBar.tsx`, `AuthContext.tsx`
Frontend API client files	`area.api.ts` in `src/lib`	`auction.api.ts`, `bundles.api.ts`
Database tables and columns	snake_case	`listings`, `module_id`, `created_at`
Environment variables	SCREAMING_SNAKE_CASE	`DATABASE_URL`, `JWT_ACCESS_SECRET`
Git branches	`type/short-description`	`feature/saved-search-frontend`, `fix/azure-fixes-and-faculty-loads`

4. TypeScript and Formatting Standards

4.1 Compiler Configuration

Both the backend and frontend use fairly strict TypeScript settings. The main ones worth knowing about:

- `strictNullChecks` is on, so you have to actually handle `null`/`undefined` instead of just assuming a value will be there.
- `forceConsistentCasingInFileNames` is on. This mostly matters because Windows/-macOS don't care about file name casing but Linux does, so an import with the wrong case can pass locally and still break the build.
- Class properties that never get reassigned should be marked `readonly`, so if you accidentally try to reassign one, the compiler catches it instead of it becoming a runtime bug.

4.2 Linting

ESLint runs in `ci.yml` on every push and PR to `main` and `develop`, separately for backend and frontend. If lint fails, the `ci-success` check fails, and branch protection won't let the PR merge until it's fixed.

> **Something we learned the hard way:** running `eslint --fix` across the whole repo mid-sprint also touches files you weren't even working on, including stuff a teammate had half-finished. Only stage and commit the files you actually meant to change, and `git checkout -- <file>` the rest instead of committing a giant auto-fix.

4.3 Static Analysis

SonarCloud also runs on every push/PR to `main` and `develop` and checks for code smells, security hotspots, and coverage. One thing that came out of an early SonarCloud finding: we now pin GitHub Actions to full commit SHAs instead of version tags in every workflow file, since tags can technically be moved.

5. Backend Standards (NestJS)

5.1 Layering

Controllers only handle HTTP stuff, routing, guards, DTO validation. All the actual logic lives in services, and services never write raw SQL, everything goes through TypeORM repositories. This way you can read a controller top to bottom and know what an endpoint does without having to dig through query logic.

5.2 Guards and Authorization

Every protected route uses `@UseGuards(JwtAuthGuard, RolesGuard)`. We don't hand-roll auth checks inside controller methods, it always goes through guards so the same check behaves the same way everywhere.

5.3 Data Transfer Objects

Every request body gets its own DTO class. The DTO is the source of truth for what an endpoint accepts, and validation decorators live there, not scattered through the service.

5.4 Database Migrations

Migrations have honestly caused us more real bugs than anything else in this project, so these rules exist because of specific things that went wrong, not just because they sounded like good practice:

- **Migration files live directly in `backend/src/database/migrations/`, never in a subfolder.** We once had a nested `migrations/src/migrations/` folder and it silently broke the compile into `dist/`, because it messed with TypeScript's implicit `rootDir` guess.
- **The migration class name has to end with the exact same timestamp as the filename.** So for `1784402940000-AddSavedSearches.ts`, the class needs to be `AddSavedSearches1784402940000`. TypeORM only checks this at runtime, not when it compiles, so it looks fine until the migration actually tries to run.
- **Don't add a `name = '...'` property to a migration class if you don't need to.** TypeORM checks that property first if it exists, and it'll silently override the correct class name even after you've already fixed the class name. This is how the same migration failed twice in a row for us, once for the class name and then again for basically the same reason in disguise.

- **Double check every column name in a migration's raw SQL against the actual entity, don't just assume.** We once wrote `module` in an index when the real foreign key column (auto-generated by TypeORM from the `@ManyToOne` relation) was actually `module_id`.
- **Run `npm run build` and check the migration actually shows up in `dist/database/migration` before assuming a fix worked.** A fix that only exists in `src/` does nothing in a deployed container, since that runs the compiled `dist/` output.
- Seed scripts (`university.seed.ts`, `faculty.seed.ts`) check if a row already exists before inserting, so you can re-run them safely against a DB that's already partly seeded without ending up with duplicates.

5.5 Event-Driven Communication Between Modules

Notifications, audit logging and saved-search matching all react to things that happen in other modules (a listing being approved, a report being filed, a new listing matching someone's saved search). Instead of the listings or reports service calling the notification service directly, the source module emits a typed event (defined in its own `events/` folder) and a listener in `notifications/` handles it. This keeps modules from depending on each other in circles, and it means adding a new reaction to an event doesn't require touching the module that emits it.

5.6 Concurrency and Transactions

Auctions introduced code where two requests can race each other, so the following rules apply to any logic like that:

- Anything that reads a value and then writes based on it, where two users could hit it at the same time (placing a bid is the main example), runs inside a manually managed TypeORM transaction (`dataSource.transaction`) with a pessimistic row lock (`SELECT ... FOR UPDATE`) scoped to the single row being contended, not the default repository `save()`.
- Do the validation *after* taking the lock, not before, otherwise the check can be stale by the time the write happens.
- Bids are append-only, including rejected ones with a reason. They are never updated or deleted, so the history stays a reliable audit trail.
- Postgres stays the source of truth. Firestore is only the broadcast layer that pushes state to connected clients, the same split the messaging feature already uses.

- Time-based transitions (an auction closing, an anti-sniping extension) are automated in the backend and never depend on a client doing something.
- Concurrency-sensitive code needs a test that fires the competing calls at the same time and asserts exactly one wins, since ordinary sequential unit tests can't catch a locking bug.

5.7 External Services and Graceful Degradation

The AI-assisted listing flow calls Azure AI Vision, which is a network dependency we don't control. The rules for anything like that are:

- Endpoints and keys come from configuration and are never hardcoded (see the secrets section).
- Every external call has a timeout, so a slow provider can't hang a request.
- The feature must fail soft. If the call errors or times out, the user falls back to normal manual entry with nothing blocked, and this fallback is tested on purpose rather than assumed.
- Output from an external service goes through the same DTO and validation as manual input, there's no parallel data path. OCR results only pre-fill the Create Listing form and the seller confirms before anything is saved.
- Messy or approximate logic on top of external output (like matching OCR text to a book) lives in small, pure utility files such as `book-matcher.util.ts` and `book-details-extractor.util.ts`, separate from the Azure-calling service, so it can be unit tested without the network.

5.8 Algorithmic Code

The bundle optimizer is a greedy heuristic for what is really a set-cover style problem. Code like this lives in a service with a narrowly typed input and result (module IDs in, a structured recommendation out), stays free of HTTP concerns so the controller stays thin, and is unit tested against its edge cases: no overlap between sellers, one seller covering everything, partial overlap, and modules with no approved listings at all.

6. Frontend Standards (Next.js)

6.1 Styling

We use Tailwind everywhere, utility classes right in the JSX instead of separate CSS files. This keeps a component's styling visible next to its markup, and using the token-based colours keeps everything on-brand with the Agile Bridge style guide instead of random one-off colours creeping in.

6.2 Authentication State

Auth state lives in one `AuthContext`, mounted once in the root layout. Which routes need login is just an explicit list (public vs. protected) checked against the current path, not a bunch of scattered per-page checks. If you add a new public page (like a help page), you have to add it to that list or it'll redirect people to login who shouldn't need to. This actually happened to us during Demo 2 prep, the Help and Brand Style Guide links in the footer were wrongly requiring login.

6.3 API Calls

All backend calls go through one shared client in `src/lib` instead of components calling `fetch` directly. That way the base URL, credentials, and error handling only need to be right in one place.

6.4 Component Organisation

Reusable UI is grouped by feature under `src/components` (`listings/`, `auction/`, `bundles/`, `messaging/`, `notifications/`, `admin/` and so on), with the generic building blocks (`Button`, `Modal`, `Input`, `Badge` and friends) in `components/ui`. A feature component that would be useful elsewhere gets moved to `ui` rather than imported across feature folders.

Screens that need a genuinely different layout on mobile and desktop (login, registration, messages) are split into separate `Desktop` and `Mobile` components under one page component, instead of piling breakpoint conditionals into a single file.

6.5 Interactive and Media Components

The AI photo capture flow and its four-corner crop editor (`AiPhotoCapture`, `CornerCropEditor`) follow the same graceful-degradation rule as the backend: show a loading state, time out sensibly, and never block the manual form if the AI step fails. Image processing helpers

(data URL conversion, background removal) are pure functions in `src/utils` with their own tests, not logic buried inside a component.

7. Git and Version Control

7.1 Branching Model

- `main` is always what's actually in production.
- `develop` is always what's on staging.
- All work happens on a feature branch off `develop`, named `type/short-description` (e.g. `feature/saved-search-frontend`, `fix/azure-fixes-and-faculty-loads`).
- In the final sprint the rule tightened: the last `develop` to `main` merge happens the day before submission, never on the morning of the deadline.
- Feature branches merge into `develop` through a PR. `develop` merges into `main` the same way, once we're sure it's working on staging.

7.2 Branch Protection

Both `main` and `develop` have branch protection on:

- No direct pushes, everything has to come through a PR from a feature branch.
- At least one other team member has to review and approve it.
- The `ci-success` check (backend + frontend lint/test jobs plus SonarCloud) has to pass, or the merge is blocked.

7.3 Commit Messages

We try to follow Conventional Commits, so `type: short description`:

Type	Used for
`feat`	A new feature
`fix`	A bug fix
`ci`	Changes to GitHub Actions workflows or pipeline config
`docs`	Documentation-only changes
`refactor`	Restructuring code without changing behaviour

> **Something we agreed on:** commit a working fix straight away instead of leaving it staged. We lost an uncommitted fix once to an accidental `git checkout` and had to redo the whole thing.

8. Testing Standards

Backend tests run with Jest against a real, temporary Postgres container that spins up inside CI, not a mocked DB connection, so a passing test suite actually means something rather than just testing logic in isolation. The backend has three layers: unit specs beside the code (`*.spec.ts`, e.g. the bundle optimizer and the OCR book matcher), integration specs in `backend/test/` that exercise services against the real database, and Supertest e2e specs (`*.e2e-spec.ts`) that hit the HTTP layer.

Frontend tests are Jest and React Testing Library unit tests kept in `tests/` folders beside the components, hooks, utils and API client files they cover (including the AI photo capture and crop editor components), plus Cypress end-to-end specs running as their own job in `ci.yml`: login (full attempt, validation errors, wrong credentials, password visibility toggle, navigation) and registration (Steps 1 to 3 of the four-step flow; Step 4's OTP verification isn't automated, since that needs the run to read the OTP from a Mailtrap inbox rather than a real email).

A few standards apply to everything new:

- A feature isn't done until its logic has tests, and its service contract and requirements paragraph are documented in the same sprint.
- Nothing demoed is mocked. If something isn't wired end to end, we ship a smaller real slice instead of a larger mocked one.
- Non-functional requirements need repeatable, executable checks with saved evidence (load tests, accessibility scans, dependency audit), run against the deployed configuration where possible.
- Concurrency-sensitive code follows the concurrent-call test rule in the backend concurrency section.

See the Testing Policy document for the full breakdown of what's covered and what isn't yet. Coverage for both sides is tracked through Codecov, with separate flags for backend and frontend.

Backend tests run with Jest against a real, temporary Postgres container that spins up inside CI, not a mocked DB connection, so a passing test suite actually means something rather than just testing logic in isolation. Frontend tests are Jest unit tests, plus a growing set of Cypress end-to-end specs running as their own job in `ci.yml`: login (full attempt, validation errors, wrong credentials, password visibility toggle, navigation) and registration (Steps 1 to 3 of the four-step flow; Step 4's OTP verification isn't automated yet, since that needs the run to read the OTP from a Mailtrap inbox rather than a real email). See the Testing Policy document for the full breakdown of what's covered and what isn't yet.

Coverage for both sides is tracked through Codecov, with separate flags for backend and frontend.

9. Environment Variables and Secrets

- No credentials, connection strings, or API keys ever get committed, in any file, at any point.
- Locally, variables go in a `.env` file at the repo root, which `docker-compose.yml` pulls in through `$VAR_NAME`. It's in `.gitignore` and never committed.
- In deployed environments, variables are stored as Azure Container Apps secrets (`secretref:`) and injected at the container level, or as GitHub Actions secrets for anything CI/CD itself needs (like `AZURE_CREDENTIALS`).
- Keys for external services (Azure Blob Storage, Azure AI Vision, Firebase, the email provider) follow the same rule as every other credential: environment variables locally, Container Apps secrets when deployed, never in source.
- A JSON-formatted secret (like the Firebase service account key) is stored as one env variable holding the whole JSON string, and parsed at runtime with `JSON.parse()`, instead of being a file, since an actual credential file should never be sitting in the repo.

Uni Textbook Marketplace

In collaboration **with:**

agilebridge
CONSIDERED
SOFTWARE SOLUTIONS

NEXUSDEV

2026 NexusDev - University of Pretoria - COS 301 Capstone Project
This document is version-controlled in the GitHub repository under
`/docs/Demo_4/Coding_Standards.pdf`